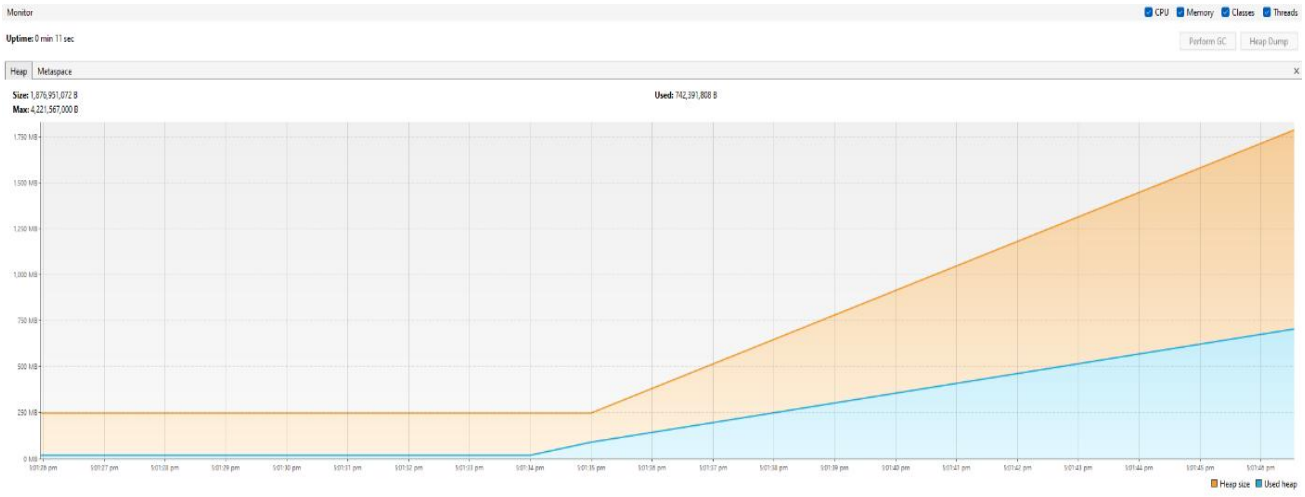
Note: All tests include a 10-second delay at the start using Thread.sleep(10000) and total test time mentioned for each test has already been adjusted to exclude this delay.

Stress Test 1 – stressTest_memAppender_velocity_arrayList_noDiscard

This is the stress test for the Memory Appender using Velocity Layout and ArrayList, ensuring there are no logs being discarded.



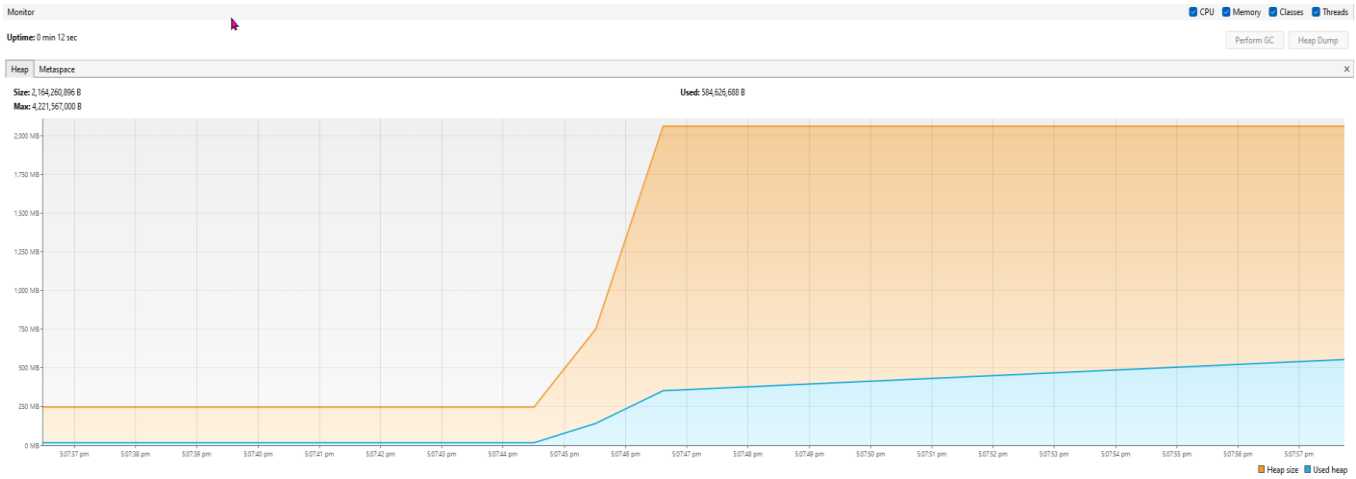
Total test time was approximately around 1 second and 178 milliseconds (approx. 2,180,295 logs per second) and total logs were 3,000,000.

- After the 10 second delay using Thread.sleep(10000), memory steadily increases due to the log accumulation.
- Memory usage grows progressively since the ArrayList accumulates up to 3,000,000 logs.
- In order to meet the memory needs as ArrayList grows, the JVM expands the heap (orange line), accommodating memory requirements.

The graph shows steady memory growth, but eventually may cause out-of-memory issues over time if logs are too large.

Stress Test 2 – stressTest_memAppender_velocity_linkedList_noDiscard

This is the stress test for the Memory Appender using Velocity Layout and LinkedList, ensuring there are no logs being discarded.



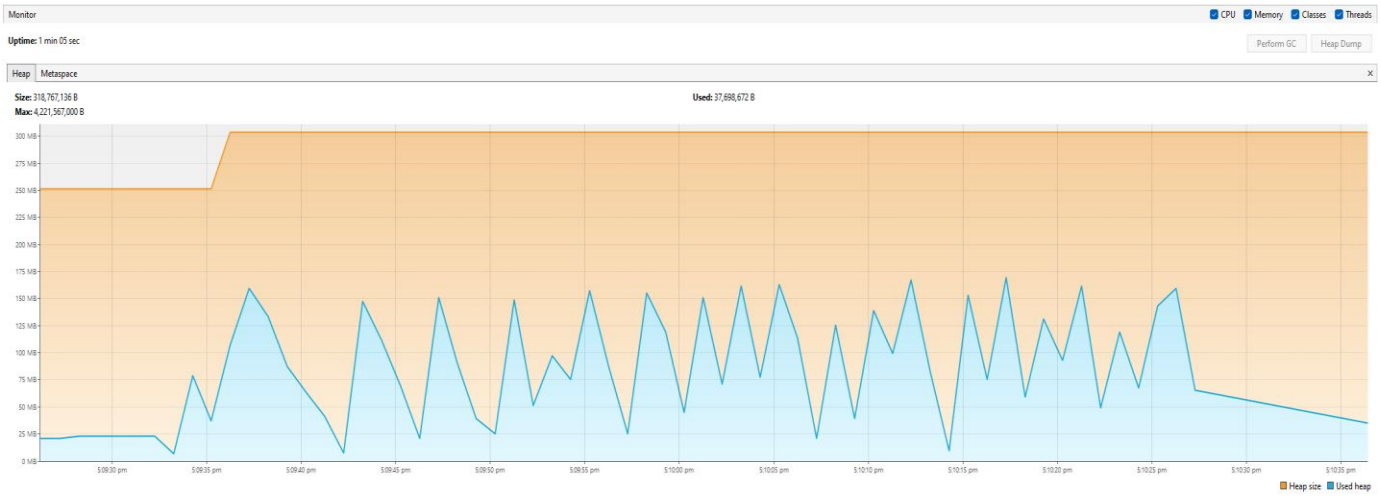
Total test time was approximately around 1 second and 794 milliseconds (approx. 1,671,913 logs per second) and total logs were 3,000,000.

- Heap usage (blue line) sharply increases after the 10-second delay since the LinkedList begins to collect logs.
- Heap size (orange line) expands quickly to meet memory requirements, and eventually stabilises once sufficient space is allocated.
- The LinkedList's overhead, which are the extra references per node, leads to a steeper memory increase compared to ArrayList, leading to greater initial memory usage.

The graph indicates LinkedList has larger memory overhead, but the memory demands then flattens when the heap has sufficiently expanded.

Stress Test 3 – stressTest_consoleAppender_velocity

This is the stress test that evaluates the Console Appender using Velocity Layout.



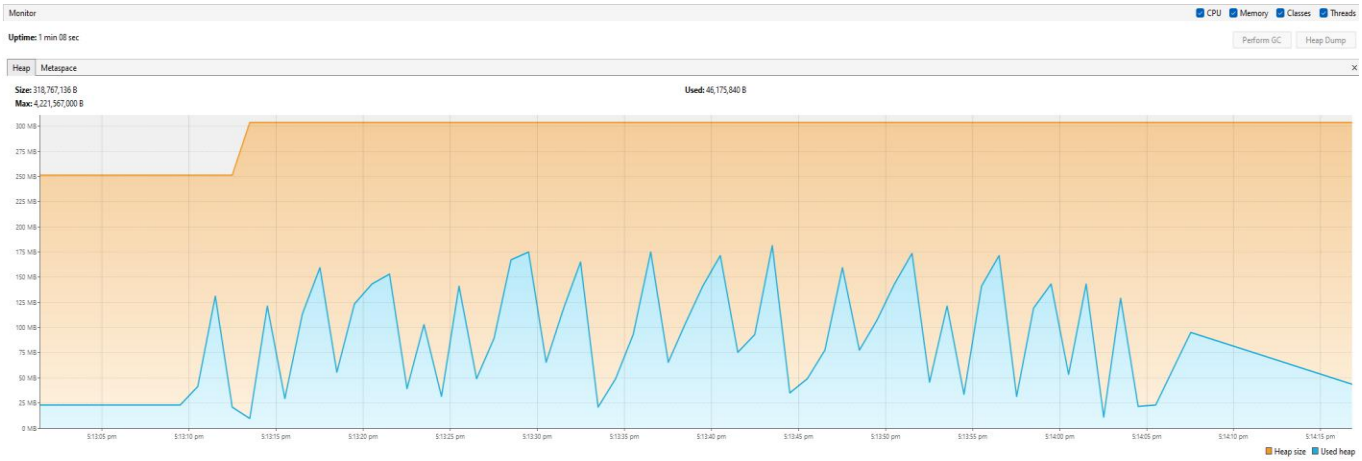
Total test time was approximately around 52 seconds and 293 milliseconds (approx. 57,368 logs per second) and total logs were 3,000,000.

- Frequent memory spikes and sharp declines in memory usage indicate the ongoing garbage collection activity.
- Memory usage peaks recurringly meaning the console output is generating temporary objects, which then promptly cleared by the garbage collection.
- Frequent and quick memory drops suggests that garbage collection works efficiently to reclaim memory quickly.
- Memory usage is lower overall since Console Appender only outputs logs without storing them, unlike ArrayList and LinkedList.

The graph illustrates that Console Appender uses less memory due to its transitory objects are efficiently managed by garbage collection.

Stress Test 4 – stressTest_fileAppender_velocity

This is the stress test that evaluates File Appender using Velocity Layout.

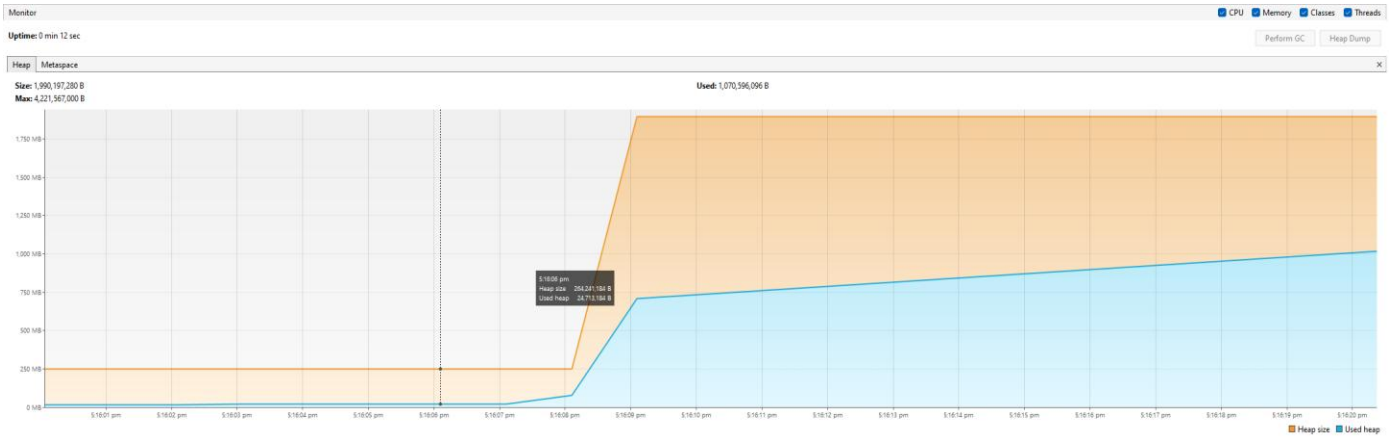


Total test time was approximately around 53 seconds and 193 milliseconds (approx. 56, 397 logs per second) and total logs were 3,000,000.

- Similar to Console Appender, the File Appender also exhibits memory fluctuations, with frequent spikes and drops, indicating active garbage collection cycles.
- Though, File Appender's peaks are slightly steadier, indicating it processes larger blocks of data that uses more memory even if then still regularly being cleared by garbage collection.
- Overall memory usage is relatively low since logs are stored on disk, minimising the memory retention.
- Writing to a file introduces I/O latency, which leads to the memory usage settle a little bit more before each garbage collection cycle occurs.

Stress Test 5 – stressTest_patternLayout

This is the stress test conducted on Pattern Layout using Memory Appender.



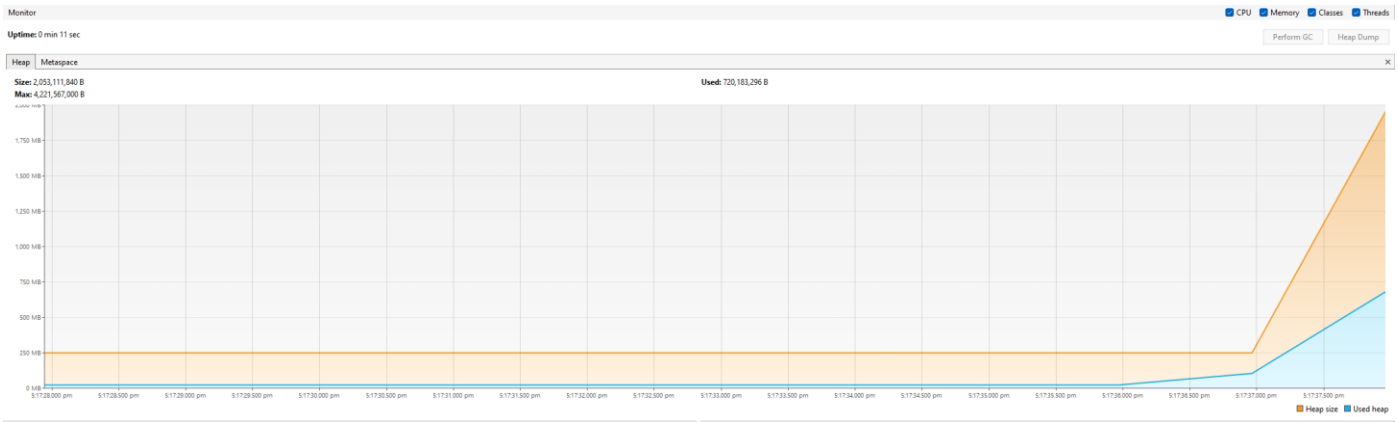
Total test time was approximately around 1 second and 551 milliseconds (approx. 3, 223, 589 logs per second) and total logs were 5,000,000.

- Following the 10-second delay, memory usage spikes rapidly, with the heap (orange line) growing to accommodate logs formatted with the Pattern Layout.
- The Pattern Layout contributes to adding extra overhead to each log entry, which leads to higher overall memory consumption.
- After the heap expands adequately, memory usage (blue line) stabilises, as all the logs are kept in memory.

The graph shows how the Pattern Layout's overhead results in higher memory usage, with the heap settling once it adapts to store the formatted logs.

Stress Test 6 – stressTest_velocityLayout

This is the stress test conducted on Velocity Layout using Memory Appender.

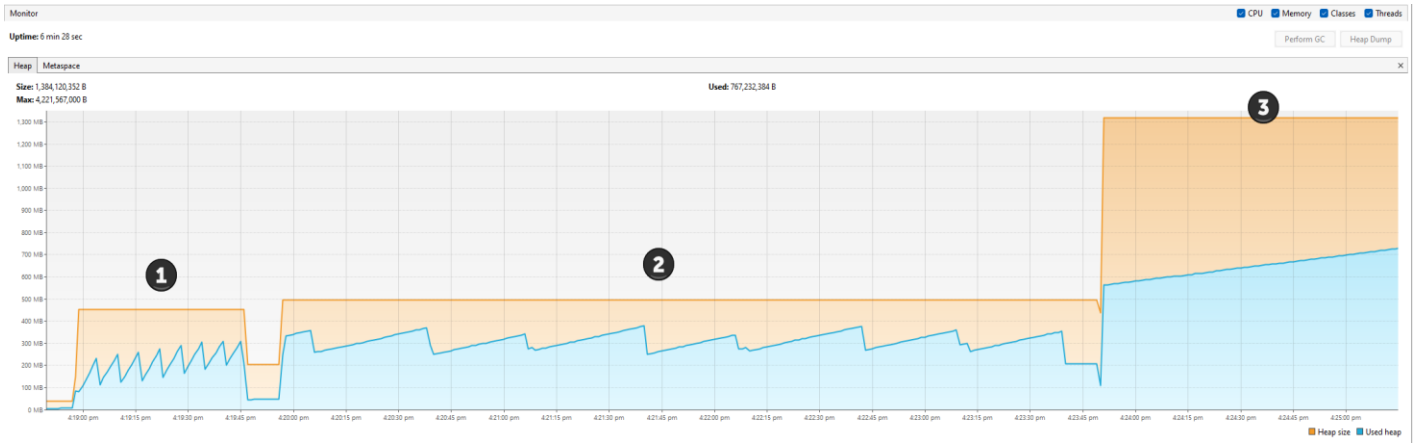


Total test time was approximately around 1 second and 579 milliseconds (approx. 3, 165, 050 logs per second) and total logs were 5,000,000.

- The graph displayed a delayed but sharp rapid rise in memory usage towards the end, suggesting that velocity layout adds a substantial memory overhead as logs accumulate.
- The heap size (orange line) increases in size late in the test, implying that Velocity Layout processing demands more memory in the later stages, potentially due to its more complex and template-based processing.

Stress Test 7 – stressTest_memAppender_velocity_arrayList_usingDiscard (Parameterised – keeping 10%, 50%, and 90%).

This is the stress test on the Memory Appender, using VelocityLayout and ArrayList, ensuring the discarded logs are accurate. It is also a parameterised test to keep 10%, 50%, and 90% of the logs while discarding the remainder.



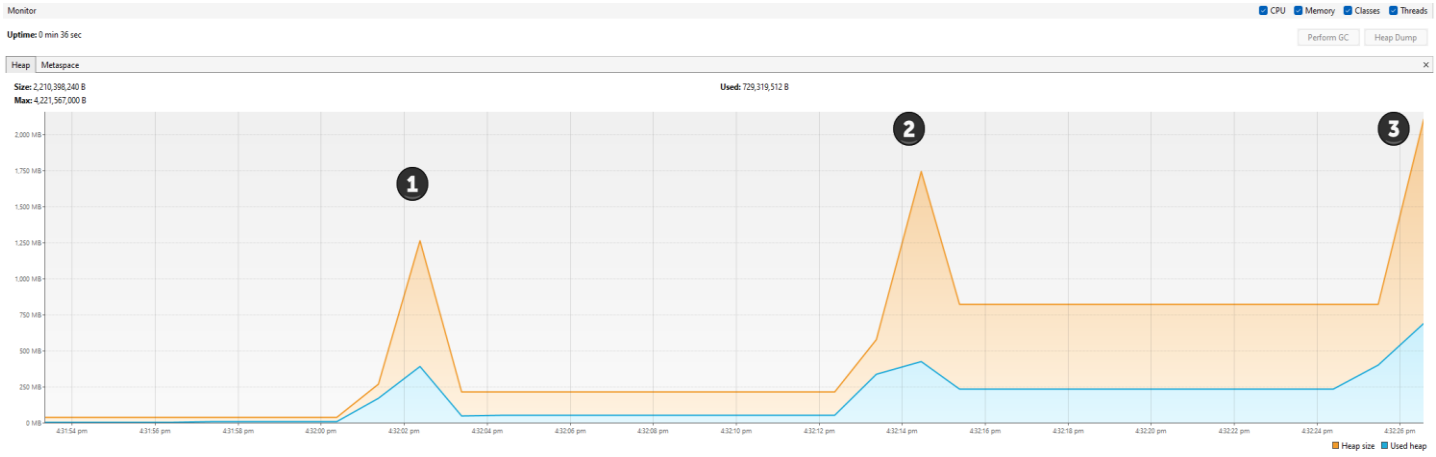
1. Keeping 10% logs (Number 1) – Approximately 47 seconds and 516 milliseconds (approx. 63, 135 logs per second)
   - With the 90% of logs being discarded, frequent garbage collection occurred. This causing overall memory usage to drop and ArrayList shrinking quickly.
   - The heap expands minimally and remains low due to the high rate of logs discarding, leading to efficient memory usage.

2. Keeping 50% logs (Number 2) – Approximately 3 minutes 37 seconds (approx. 13, 763 logs per second)
   - With 50% logs retained, the ArrayList grow more often, resulting in a slow garbage collection cycle.
   - The ArrayList resizes more often, which is time-consuming as it involves creating new arrays and data copying.
   - 50% retention is too large for quick processing (like with 10%), but too small for fast garbage collection (like with 90%), leading to slower execution.

3. Keeping 90% logs (Number 3) – Approximately 1 minute 24 seconds (approx. 35, 302 logs per second)
    - With 90% of logs being retained, it causes a sharp increase in memory usage, resulting in significant heap expansion.
    - Fewer discarded logs lead to garbage collection occurs less often, therefore, causes the larger heap size.
    - The ArrayList also requires larger memory blocks during its resizing, hence the surge in memory usage as more logs are retained.

Stress Test 8 – stressTest_memAppender_velocity_linkedList_usingDiscard (Parameterised – keeping 10%, 50%, and 90%).

This is the stress test on the Memory Appender, using VelocityLayout and LinkedList, ensuring the discarded logs are accurate. It is also a parameterised test to keep 10%, 50%, and 90% of the logs while discarding the remainder.



1. Keeping 10% logs (Number 1) – Approximately 1 second and 492 milliseconds (approx. 2, 010, 485 logs per second)
    - LinkedList has a sharper memory spike due to its overhead from node pointers. It also handles insertions and deletions more efficiently compared to ArrayList, especially when many elements are discarded, making it quicker in this part.
    - With only 10% logs retention, it causes a quick memory spike, but then promptly reclaimed, making the test so much quicker compared to ArrayList.

2. Keeping 50% logs (Number 2) – Approximately 1 second and 948 milliseconds (approx. 1, 539, 583 logs per second)
    - LinkedList is faster as well in this part due to the efficient insertions and deletions compared to ArrayLists, which needs resizing and shifting elements.
    - Even with the steep memory spike from the LinkedList's structural overhead, executions remain faster because of its efficient log retention and discarding.

3. Keeping 90% logs (Number 3) – Approximately 1 seconds 748 milliseconds (approx. 1, 715, 422 logs per second)
    - With 90% retention, it causes even larger memory spike, but again, LinkedList handles memory more efficiently than ArrayList in this case because it avoids the need to resize and copy large memory blocks, enabling a quicker test finish even with high memory usage.

The graph shows how LinkedList, despite the steeper memory usage caused by its structural overhead, it outperformed ArrayList in log retention and discarding, especially at 50% and 90% retention, where the ArrayList's resizing becomes the limiting factor causing bottlenecks.